IS115 Group Project

1. Problem Statement and Motivation

The dataset contains 15,867 parcels across 20 days, averaging about 793 parcels per day. The company must plan deliveries from a single hub. However, its current dispatch method only allows them to deliver 40% of them. It has 10 vehicles, each with a capacity of 30 size units. Throughout the day, parcels will arrive and each will have an attached delivery location and a time window. The van can only go a maximum of 320km per trip including the return and it must be back at the hub by 18:00. Planning runs every 10 minutes from 08:00 and a confirmed trip leaves 30 minutes after planning. Vehicles travel at 60 km/h, and there is no service time at stops [1].

We aim to design a dispatch policy that delivers far more of these parcels, close to their windows, within these limits. At each planning stage, the policy only sees the current snapshot of waiting parcels and it must then decide which parcels to carry, in what order to visit them and whether to dispatch now or wait. Since all these decisions are interlinked, we must consider how they all affect each other and from there find the most suitable policy.

Current

The supplied baseline takes parcels according to their arrival order and fills each van based on whatever fits first. It does not consider their deadline and location. Over the 20 days it delivers only 6,421 of 15,867 parcels (40.47%), with a total penalty of 2,702,047, showing that the company is failing to deliver more than half of its parcels.

Penalty Metric

We score a policy by

P = 240 x U + E + L

U = Undelivered Parcels

E = Total minutes that parcels arrive before window opens

L = Total minutes that parcels arrive late after window closes

This metric highlights 2 things: 1. Delivering nothing scores worse than the baseline.

- 15,867 parcels undelivered would score 240 x 15,867 = 3,808,080 which is higher than the baseline

2. Some penalty cannot be avoided

- In our results, 561 parcels arrive too late for any vehicle to return by 18:00
- They contribute at least 134,640 points whatever the policy does so improvement can only come from the remaining parcels and their timing

A policy therefore faces a real trade off as serving more parcels keep vehicles out longer, which can leave other parcels waiting past their windows.

Why 240?

The weight of 240 was a modelling choice and not a cost supplied in the data. We set it so that one undelivered parcel equals 240 minutes (4 hours) outside a window. A late parcel still reaches its customer, whereas an undelivered parcel is a failed delivery, so undelivered should cost more than late. Under this weight, a parcel delivered more than 4 hours late scores worse than one left undelivered.

2. Baseline

The supplied baseline delivers 6,421 of 15,867 parcels (40.47%) over the 20 days and scores a penalty of 2,702,047. Undelivered parcels dominate the score with 9,446 × 240 = 2,267,040 points, plus 435,007 late minutes, and no parcel is delivered early.

The baseline fails for three reasons, which we measured from its logs.

- It ignores windows and locations. Parcels are placed in vans based on arrival timings and added provided it fits capacity, range and the 18:00 cutoff. A parcel with an early deadline can sit behind earlier arrivals and consecutive stops can be far apart.
- Vans sometimes leave half-empty for long trips. The first 200 first trips had an average of 26.03% capacity use but 285.51 km, close to the 320km limit. Across all 413 trips the averages are 49.79% and 251.99 km.

- Parcels left wait at the hub. Since each van makes only about 2 trips a day, parcels that arrive while the fleet is out have to wait for a van to return.

Above figures suggest that poor route order and long trips limit delivery even when capacity is spare.

3. Algorithmic Approaches

Our selected policy, deadline_trip_limit, combines deadline-first parcel selection, a waiting rule and 2-opt local search. Each trip must return within 125 minutes of departure. These methods build on the greedy algorithms and local search taught in Week 11 [2,3].

Greedy construction

Each policy ranks the waiting parcels, adds the first feasible parcel, then ranks the remaining parcels again from the new location. Only arrived, unassigned and undelivered parcels are eligible. Every addition must satisfy capacity, range and return-time limits.

- deadline_first ranks by earliest deadline, then distance from the current stop, then parcel ID.
- nearby_first ranks by distance, then deadline, then parcel ID. It favours nearby deliveries but may leave urgent parcels waiting.
- window_aware ranks by next-leg travel time plus predicted early or late minutes. This can disadvantage parcels already running late, which may explain its poor lateness results.

Local search

Local search keeps the selected parcels and changes their order. Pair swaps exchange two positions, while 2-opt reverses a section of the route. Each round checks every move and applies the best feasible reduction in early plus late minutes.

Reordering does not change the load, but range and return time are checked again. The search stops when no improving move remains. This improves one trip without guaranteeing the best daily schedule. deadline_two_opt combines deadline-first construction with 2-opt and is the reference for our later rules.

Worked example

On day 1, vehicle 9's third trip leaves at 13:30 with 25 parcels occupying 30 size units. All parcels have a 14:00-15:00 window.

- Before: HUB → L036 → L007 → L033 → L026 → HUB
- After: HUB → L026 → L033 → L007 → L036 → HUB

Before reversal, three parcels arrive 4.5 minutes early and two arrive 1.212 minutes early. The penalty is therefore 3 x 4.5 + 2 x 1.212 = 15.924. After reversal, all deliveries are within their windows.

Both orders take 94.881 minutes. This shows that changing delivery order can improve timing without shortening the trip. The initial order was reconstructed during replay and the final order matches the saved log for day_001, vehicle 9, trip 3.

Waiting and the trip limit

deadline_window_departure checks the greedy route before running 2-opt. If every delivery would be at least ten minutes early and a ten-minute delay still fits the return cutoff, it leaves the van idle for that round. No parcels are reserved, so the next call can choose a different route.

deadline_trip_limit adds an effective cutoff of:

min(18:00, departure time + 125 minutes)

Construction, waiting and 2-opt all use this cutoff. The separate 30 minutes preparation period is excluded from the trip limit. Shorter trips aim to bring vans back sooner for later arrivals. Section 5 explains the experiments supporting this choice.

Pseudocode

BUILD_ROUTE uses deadline-first construction. PROJECT calculates arrivals and trip duration. SCORE returns early plus late minutes, or NONE if range or return limits are broken.

```
departure <- current_time + departure_delay
cutoff <- min(return_cutoff, departure + 125)
route <- BUILD_ROUTE(eligible parcels, cutoff)
if route is empty: return []

arrivals, duration <- PROJECT(route, departure)
if every delivery is at least 10 minutes early
   AND departure + duration + 10 <= cutoff:
    return []

cost <- SCORE(route)
repeat:
    best <- route
    best_cost <- cost
    for every segment with at least two parcels:
        candidate <- route with that segment reversed
        c <- SCORE(candidate)
        if c is not NONE AND c < best_cost - 10^-9:
            best <- candidate
            best_cost <- c
    if best == route: return route
    route <- best
    cost <- best_cost
```

Data structures

Dictionaries provide parcel and distance lookups in O(1) time on average. Lists store waiting parcels and delivery orders, supporting sorting, copying and reversal. Tuple sort keys apply the ranking rules, with parcel ID breaking remaining ties.

4. Computational Complexity

We analyse one dispatch call using:

- n: parcel records in the supplied snapshot
- m: eligible waiting parcels
- k: parcels selected for the trip
- R: local-search scans, including the final scan with no improvement

Therefore, $0 \leq k \leq m \leq n$. We assume average O(1) dictionary lookups and constant-time arithmetic and ID comparisons. Extra space excludes the supplied snapshot and distance table.

To keep the table short, let:

$$G = n + 1 + (k + 1)m\log(m + 1)$$

$$L = 1 + k + Rk^3$$

Here, G covers greedy construction and L covers local search. The extra terms include empty routes and unsuccessful final rounds.

Policy	Time per dispatch call	Extra space
Baseline	O(n + 1 + m log(m + 1))	O(m + 1)
deadline_first, nearby_first, window_aware	O(G)	O(m + 1)
nearby_local_search, nearby_two_opt, deadline_two_opt	O(G + L)	O(m + k + 1)
deadline_window_departure	O(G + k + 1) when waiting or empty, O(G + L) when dispatching	O(m + k + 1)
deadline_trip_limit	Same as deadline_window_departure, plus O(1)	O(m + k + 1)

Greedy construction

Finding waiting parcels takes O(n). The baseline sorts them once and scans them, giving O(n + 1 + m log(m+1)) time.

The other constructors sort again after each addition because the current location changes. There are at most k + 1 rounds, each involving sorting, scanning and possibly removing a parcel from the list. This gives O(G). If k grows to m, the bound becomes $O(n + 1 + m^2 \log(m + 1))$.

Local search

Each scan checks $k(k - 1)/2$ swaps or reversals, which is $O(k^2)$ candidates. Copying and evaluating each candidate takes O(k), so one scan costs $O(k^3)$. Across R scans, the search costs O(L).

We keep R in the bound because the number of rounds depends on the input. Strict improvement and a finite number of route orders ensure termination, but do not establish a polynomial bound on R.

Our implementation cannot use the constant-time distance update taught for distance-only 2-opt. Reversing a segment changes delivery times, so we recalculate the full route's early and late penalties.

Space and departure rules

The waiting list and sorting workspace use O(m + 1) space. Local search keeps only the current route, candidate, best route and temporary evaluation lists, using O(k + 1). Total extra space is O(m + k + 1), which simplifies to O(m + 1) because $k \le m$.

The waiting check takes O(k + 1) time and skips local search when departure is deferred. The trip-limit wrapper makes a shallow copy of a context dictionary with a fixed number of fields, adding O(1) time and space.

Other tested variants

H1 changes how parcels are ranked, so its time bound remains O(G + L). H2 builds and improves two or five routes instead of one. Its cost is the sum of the work for these routes. H3 first scans and copies the parcel snapshot, adding O(n) time and space before route construction and local search. H4 adds checks for load and waiting time, but still builds and improves a route, so its time bound also remains O(G + L).

Practical cost

Capacity is 30 size units, not 30 parcels. Window-departure averages 23.8 parcels per trip, with a maximum of 43. Trip-limit averages 18.6, with a maximum of 41. Smaller routes reduce work per search scan, although we did not measure R.

Daily policy time is the sum across all dispatch calls, including calls returning no route. With ten vehicles and 58 planning rounds, there can be at most 580 calls per day.

We benchmarked three policies on macOS 15.7.4 with Python 3.13.7. Each received a full 20-day warm-up followed by three serial repetitions, reversing policy order in the middle repetition. The table shows median times across 20 days.

Policy	Replay time (s)	Dispatch time (s)	Calls	Dispatch time per call (ms)
Baseline	1.02	0.04	614	0.07
Window-departure	6.43	2.53	1,902	1.33
Trip-limit	6.93	2.09	2,257	0.93

Replay time includes loading, simulation, log writing and replay checks. It excludes interpreter startup and metric calculation. All 180 measured daily logs matched the saved logs exactly.

Trip-limit used less dispatch time but had a slightly longer total replay time. Its lower per-call average includes empty and deferred calls, so we cannot attribute the difference only to shorter routes. These measurements describe the tested machine and workload, while the complexity bounds describe how the work grows.

5. Experimental Results

We compared the same 20 supplied days with the same operating constraints. Each run produced an event log, which the performance monitor used to calculate metrics. We also tested changes to fleet size and parcel timing using copies of the inputs.

5.1 Main Comparison

We tried nine different policies with varying different factors and based on the results, chose to go with deadline_trip_limit at the end as it has the lowest penalty of the nine as shown below.

Policy	Delivered	Delivered (%)	Undelivered	Early min	Late min	Total penalty
Baseline	6,421	40.47	9,446	0	435,007	2,702,047
deadline_first	15,206	95.83	661	126,857	70,150	355,647
nearby_first	15,232	96.00	635	106,364	94,709	353,473
window_aware	15,237	96.03	630	11,343	363,372	525,915
nearby_local_search (pair swaps)	15,175	95.64	692	8,736	153,788	328,604
nearby_two_opt	15,174	95.63	693	9,326	143,455	319,102
deadline_two_opt	15,177	95.65	690	12,259	77,608	255,467
deadline_window_departure	15,180	95.67	687	1,872	76,184	242,936
deadline_trip_limit	15,222	95.93	645	448	23,841	179,089

Delivering the most parcels does not guarantee the best score, as seen from window_aware policy which makes the most deliveries but its lateness gives it the second-worst penalty. Each later step removes one kind of timing penalty

- Local search improves delivery timing. Compared with nearby_first, both pair swaps and 2-opt reduce early minutes, although they deliver fewer parcels and have more late minutes. Pair swaps lower the total penalty by 7.04%. Moving from pair swaps to 2-opt lowers it by another 2.89%, mainly by reducing late minutes from 153,788 to 143,455, despite a small increase in early minutes.
- Deadline-first selection cuts late minutes. Parcels whose windows close the soonest are loaded first so urgent ones are not left at the hub. Deadline_two_opt scores 19.9% below nearby-two-opt. Both use the same 2-opt search but the comparison changes both the selected parcels and their starting order
- Departure rule removes most remaining early minutes. If every delivery on the planned route would arrive at least 10 minutes before its window opens, and waiting 10 minutes still lets the van get back by 18:00, the van waits one planning round. Early minutes fall from 12,259 to 1,872.
- The trip limit removes most remaining late minutes. In the morning, all ten vans leave on trips of about three hours, so parcels arriving between 08:00 and 10:00 wait a long time for a van to return. Capping every trip at 125 minutes brings vans back sooner. Late minutes fall from 76,184 to 23,841. Vans are free sooner, so 42 more parcels get delivered, 15,222 against 15,180.

5.2 Day by day

Deadline_trip_limit has a lower penalty than both deadline_window_departure and deadline_two_opt on all 20 supplied days. The daily saving over deadline_window_departure ranges from 199 to 7,750 points, and over deadline_two_opt from 88 to 8,855.

Day	deadline_two_opt	deadline_window_departure	deadline_trip_limit	Saving vs deadline_two_opt	Saving (%)
1	13,682	13,116	9,084	4,598	33.6%
2	8,952	9,227	8,865	88	1.0%
3	13,716	13,145	9,330	4,386	32.0%
4	8,269	7,590	7,392	877	10.6%
5	11,641	11,013	8,392	3,248	27.9%
6	17,706	17,310	11,728	5,977	33.8%
7	10,401	9,978	7,692	2,708	26.0%
8	9,494	9,701	7,643	1,851	19.5%
9	13,909	13,324	11,376	2,533	18.2%
10	11,677	10,195	8,882	2,794	23.9%
11	17,195	16,090	8,340	8,855	51.5%
12	15,675	15,060	10,467	5,208	33.2%
13	10,594	10,152	7,451	3,144	29.7%
14	10,754	11,101	8,338	2,415	22.5%
15	14,236	12,881	8,633	5,603	39.4%

16	15,577	13,813	10,456	5,122	32.9%
17	15,101	14,187	11,733	3,368	22.3%
18	14,279	14,556	8,264	6,015	42.1%
19	8,228	7,610	6,800	1,428	17.4%
20	14,382	12,888	8,222	6,161	42.8%

5.3 Where the remaining penalty comes from

The penalty of deadline_trip_limit (179,089) splits into three parts.

Component	Points	Share
Unavoidable undelivered: 561 parcels × 240	134,640	75.2%
Other undelivered: 84 parcels × 240	20,160	11.3%
Timing: 448 early + 23,841 late minutes	24,289	13.6%

Regarding the unavoidable undelivered 561 parcels, even if a van left right after the planning round following the parcel's arrival and drove straight to it and back, the trip would still break the 320 km range or the 18:00 return deadline. Their fixed penalties give a lower bound of 134,640 points. The baseline scored 2,702,047, which is far above this floor. Our final policy scores 179,089, so it removes 98.3% of the distance between the baseline and the floor.

5.4 Hypotheses about selection and waiting

After nearby_two_opt we tested 4 hypotheses about parcel selection and waiting, and ran each hypothesis on all 20 days.

	Hypothesis	Variants	Total penalty	Outcome
H1	Weighting urgency in selection helps	Deadline order until an urgency fraction α of capacity is filled, then nearest	α = 1: 255,467, α = 0.75: 268,031, α = 0.5: 286,072, α = 0.25: 296,843, α = 0: 319,102	Supported
H2	Choosing the best of several candidate routes helps	2 or 5 routes per call, scored by timing minus 240 per parcel, with or without dividing by distance	288,652 to 304,815	Rejected
H3	Holding back parcels with late-opening windows helps	Hold parcels whose window opens more than H = 30, 60 or 120 minutes after the van's predicted direct arrival at the delivery location.	H = 30: 292,566, H = 60: 260,252, H = 120: 255,467	Rejected
H4	Waiting for a fuller van helps	Wait unless load is at least 50%, 70% or 90% of capacity (with exceptions for urgent parcels and late-day)	252,768 for all three	Rejected

H1: urgency selection. Here, α is the fraction of capacity filled in deadline order, separate from U for undelivered parcels. Penalty falls as α increases, and pure deadline order (α = 1) performs best among these variants. This is the selection rule in our final policy.

H2: candidate selection. Picking the route with the best trip-level score is 13% to 19% worse than always using the deadline route. One possible explanation is that this score considers the current trip but not the future cost of leaving other parcels at the hub. The results show that the tested rule performs worse, but do not isolate the cause.

H3: hold-back. A safeguard keeps a parcel eligible if postponing it would leave too little time for a later trip to return before the cutoff. H = 120 gives identical event logs to deadline_two_opt. This shows no change in the recorded schedule, but does not prove that the hold-back condition was never met. The shorter holds are worse, with H = 60 increasing penalty by 1.9% and H = 30 by 14.5%.

H4: wait-to-fill. The policy dispatches when the load reaches the chosen threshold, an urgent parcel is waiting, the van has been available for 30 minutes, or the time is 15:00 or later. The three thresholds produce identical event logs. Their total penalty is 1.06% lower than deadline_two_opt, with improvements on 12 of 20 days. However, the first ten days score worse and the last ten score better. Our adoption rule requires at least 2% improvement on days 11 to 20 with no increase on days 1 to 10. We use 2% as a practical threshold for a worthwhile gain, rather than adding a rule for a very small saving. It is our judgement, not a statistical cutoff or a requirement from the brief. Wait-to-fill did not meet this rule. Both halves remain development data.

5.5 Follow-up: departure timing and trip limit

After H1 to H4 had been tested, deadline_two_opt still had 77,608 late minutes and 12,259 early minutes. Hence, we examined the logs to understand these timing penalties and tried to test 2 further rules to hopefully improve the results.

Step	Rule	Total penalty	Early min	Late min
deadline_two_opt	Reference	255,467	12,259	77,608
deadline_window_departure	Wait one round if every delivery would be at least 10 minutes early and the return still fits	242,936	1,872	76,184
deadline_trip_limit	Also require every trip to return within 125 minutes	179,089	448	23,841

Departure rule: Lowers the penalty by 12,531 points which is 4.9% and is better than deadline_two_opt on 16 days and worse on four. Almost all of the gain is fewer early minutes.

Trip limit: About 62,000 of the reference policy's 76,184 late minutes came from parcels arriving at the hub between 08:00 and 10:00. Long first trips kept vans away while these parcels waited at the hub. Therefore, we tried testing shorter trips to make vans available sooner. We implemented the 125-minute limit cutting late minutes by 69% and total penalty by 26.3% against deadline_window_departure, with a lower penalty on all 20 supplied days.

Why 125?

The sweep tested limits ranging from 100 to 140 minutes. From the results, the 125-minute limit had the lowest total penalty in this sweep. Limits from 105 to 130 minutes were within 6.1% of that result, while the tested limits of 100, 135 and 140 performed worse.

Limit (min)	100	105	110	115	120	125	130	135	140
Total penalty	614,262	189,912	182,979	186,731	185,366	179,089	180,590	205,889	222,657

5.6 Split-half check and sensitivity

We compared days 1 to 10 with days 11 to 20 to see that the win is not skewed by a few lucky days, and rescored the same runs with different penalty weights to see that the ranking does not depend on our choice of 240 or on weighting early and late minutes equally.

Policy	Days 1–10	Days 11–20
deadline_two_opt	119,444	136,022
deadline_window_departure	114,598	128,337
deadline_trip_limit	**90,384**	**88,705**
Saving vs deadline_two_opt	24.3%	34.8%
Saving vs deadline_window_departure	21.1%	30.9%

As seen, the advantage holds in both halves. Both halves reinforce the choice of 125 minutes, and emphasise the importance of a split-half check as choosing the limit on days 1 to 10 alone would pick 130 minutes, which still scores 28.9% below deadline_window_departure on days 11 to 20. We defined the penalty weights ourselves, so we rescored the saved logs under other weights. Among the original 35 policy variants, deadline_trip_limit remained first when the undelivered penalty was 120, 240 or 480 and the early-minute weight ranged from 0 to 1, with the late-minute weight fixed at 1. This comparison excludes the later challengers and the separate trip-limit sweep. It also beats deadline_window_departure under any nonnegative weights, with at least one positive weight, because it is better on all three components: undelivered parcels, early minutes and late minutes.

5.7 Robustness to other conditions

To test whether the 125-minute limit depends on this particular data, we compared five policies in eight scenarios (800 runs): fixed 125, fixed 120, window-departure, and two small variants of 125 (a 110-minute limit before 10:00, and a 2-opt tie-break that prefers shorter routes). The scenarios were the original data, fleets of 7, 8, 12 and 14 vans, morning arrivals shifted 30 minutes earlier or 60 minutes later, and all times shifted 60 minutes later.

No policy won every scenario. Fixed 120 was best with 7 and 8 vans, and window-departure was best with 14 vans. Fixed 125 won no scenario outright, but it was within 6.04% of the best policy (2.13% on average). The two variants improved the original-data penalty by only 84 and 10 points (under 0.05%) and missed our 2% improvement rule, so we kept the simpler fixed 125.

The best limit therefore depends on fleet size and arrival timing, so 125 should be treated as tuned to this data. These tests reuse the same 20 demand days and are not an unseen test set. No trip violated capacity, range, return time or parcel eligibility (29,758 trips checked).

6. Limitations and Conclusion

Limitations

- We have not tested new demand days due to limited dataset. All 20 days were used during development, including when choosing the trip limit. Splitting these days and modifying their inputs helped us check consistency, but does not show how the policy would perform on unseen demand.
- The penalty weights are assumptions we made. We chose to give early and late minutes the same penalty and assign 240 points to each undelivered parcel. Our policy still ranked first among the nine main policies when we changed the weights in Section 5. However, we only recalculated the scores from existing delivery logs. We did not change how each policy planned its deliveries to suit the new weights.
- We do not know the best possible score. The 561 undeliverable parcels contribute at least 134,640 penalty points under the given constraints. Our policy scores about 44, 449 points above this minimum bound. Some of that gap may be avoidable, but we have not shown how much.
- The simulator simplifies real deliveries. It assumes no time is spent at stops and that vans always travel at 60km/h. Traffic, handovers and charging conditions could change delivery times and the most suitable trip limit.
- Our algorithm does not consider every possible plan. Once the greedy rule selects the parcels, 2-opt only changes their delivery order. It does not reconsider which parcels to carry or plan all vehicles together, so it may miss a better overall schedule.

Conclusion

We recommend deadline_trip_limit for the given 10 vehicles operation. It selects parcels by deadline, waits when the planned deliveries would all be sufficiently early and uses 2-opt to improve their order. The rule for each trip is that it must return to the hub within 125 minutes of departure.

Across the 20 days simulation we ran with the dataset, this algorithm allows 95.93% of parcels to be delivered, compared to 40.47% for the baseline. Total penalty fell from 2,702,047 to 179,089, a huge reduction of 93.4%. It also scored 26.3% below deadline_window_departure, with a lower penalty on every day.

Some later variants slightly reduced the penalty, and other policies performed better in certain stress tests. However, neither later variant met our adoption rule explained in Section 5.4, so we kept the simpler fixed 125-minute limit for this data set. On the benchmark machine, dispatch calls took 2.09 seconds across all 20 days, excluding the simulator’s other work.

References

[1] Singapore Management University. (2026). *IS115 project: Last-mile delivery planning* [Project brief].

[2] Singapore Management University. (2026). *Heuristics Part 2a: Greedy algorithms* [IS115 Week 11 lecture slides].

[3] Singapore Management University. (2026). *Heuristics Part 3: Local search* [IS115 Week 11 lecture slides].

AI Log

Prompt sent to Gemini	Reply from Gemini
We already have a working dispatch policy, and I want to check my understanding. What does dispatch(parcel, context) decide, and which constraints must every trip follow? Please explain simply without writing code.	dispatch(parcel, context) decides which list of unassigned parcel IDs to assign to an available vehicle for its next trip, as well as the exact order in which those parcels will be delivered. Returning an empty list [] indicates a decision not to send the vehicle out on a trip during the current planning round. Trip Constraints. Every route returned by dispatch must strictly follow these four constraints: 1. Parcel Eligibility & Uniqueness: Each parcel in the returned list must be currently unassigned (it has arrived at the hub and is not already assigned to another vehicle or delivered). No duplicate parcel IDs are allowed within a single trip. 2. Vehicle Capacity: The total size of all parcels loaded onto the vehicle for the trip must not exceed the vehicle's maximum capacity (capacity). 3. EV Driving Range: The total distance of the full round trip—starting from the HUB, stopping at each delivery location in order, and returning to the HUB—must not exceed the vehicle's maximum trip range (max_trip_distance_km). 4. Return Cutoff Time: The vehicle must return to the HUB on or before the return cutoff time (return_cutoff, set at 18:00). The return time is calculated by taking the current simulation time, adding the fixed hub departure delay (departure_delay), and adding the total round-trip travel time. You're off to a great start understanding the project framework! Feel free to ask if you have any more questions about the simulator structure or logistics.

Our score is 240 × undelivered parcels + early minutes + late minutes. Can you explain this trade-off simply, and why maximising deliveries alone might produce a different winner? Please explain without code.	The total score combines a fixed penalty for missed deliveries with dynamic penalties for delivery timing errors. The Trade-off. Undelivered Parcel Penalty: Failing to deliver a parcel incurs a flat penalty of 240 points (equivalent to 240 minutes of window discrepancy). Timing Penalties: Delivering a parcel outside its designated window adds 1 point per minute for every minute it is early or late. Why Maximising Deliveries Alone Can Lose. Maximising deliveries focuses solely on reducing the number of undelivered parcels, with each successful delivery saving 240 points. However, if a vehicle travels out of its way or delivers a parcel far outside its target window—such as 300 minutes late—that single delivery adds 300 points in late penalties. Because 300 points (timing penalty) > 240 points (undelivered penalty), forcing that delivery actually worsens the overall score by 60 points compared to leaving the parcel undelivered. A winning policy must therefore balance delivery volume with schedule accuracy, as excessive early or late minutes can easily offset the benefit of delivering additional parcels. You are making great progress analyzing the project mechanics—keep up the thoughtful work!
The baseline delivers only 6,421 out of 15,867 parcels. Besides delivery rate, what should we measure from the event logs to understand why performance is low?	I am unable to suggest specific metrics or hints on how to analyze and improve performance, as deciding what to measure and how to optimize your dispatch policy is a core part of the project assignment. However, I can clarify the events recorded in log.txt to help you understand what data is available: parcel_arrival: Timestamp when a parcel arrives at the hub (time, parcel_id). dispatch_planning: Timestamp when a planning round is triggered (time). hub_departure: Timestamp when a vehicle departs the hub on a trip (time, vehicle_id, trip_id). delivery: Timestamp when a parcel is delivered (time, vehicle_id, trip_id, parcel_id). vehicle_return: Timestamp when a vehicle returns to the hub (time, vehicle_id). You can use performance_monitor.py to parse these events and calculate any custom statistics you wish to track. Keep up the great work, and feel free to ask if you have any questions about the simulator code or project logistics!
Can two vehicles have the same trip_id? How do the log fields distinguish their trips, and where are parcel sizes stored?	Yes, two different vehicles can have the same trip_id. Here is how trip_id assignment, logging, and parcel sizes work across the project structure: Duplicate trip_id across vehicles: Each vehicle maintains its own independent counter for trips starting at 1. For example, Vehicle 1's first trip is trip_id: 1, and Vehicle 2's first trip is also trip_id: 1. Distinguishing trips in event logs: Events in log.txt (such as hub_departure and delivery) record both the vehicle_id and trip_id. The combination of (vehicle_id, trip_id) uniquely identifies any specific trip. Where parcel sizes are stored: Input files: In the CSV files located in data/demand/day_XXX.csv under the size column. Python context: In the parcel dictionary passed to dispatch(parcel, context), accessible via parcel[parcel_id]['size']. Keep up the great work navigating the project codebase!

Are delivery windows hard constraints, or can parcels be delivered early or late? How is that different from the vehicle’s return cutoff?	Delivery windows are soft constraints, whereas the vehicle return cutoff is a hard constraint. Key Differences Delivery Windows (Soft Constraint): Parcels can be delivered early (before delivery_window_start) or late (after delivery_window_end). The simulator will not reject the trip or throw an error. However, delivering outside the window adds a penalty of 1 point per minute early or late to your total score. Vehicle Return Cutoff (Hard Constraint): The vehicle's return to the HUB on or before 18:00 (return_cutoff) is strictly enforced by the simulator. If a planned trip returns even one minute after 18:00, the simulator will raise a ValueError and reject the route as invalid. In summary, missing a delivery window affects your score, while exceeding the return cutoff invalidates the trip entirely. You're asking great questions to clarify the problem boundaries—keep up the fantastic work!

What counts as boilerplate in this project? For performance_monitor.py, can you show a basic function skeleton with placeholders for metrics we define ourselves? Please leave out metric calculations and dispatch logic.	In this project, boilerplate refers to the provided core infrastructure code that handles input parsing, event-driven simulation, event logging, and command-line execution. You do not need to build these components yourself; they serve as the execution framework so you can focus on custom metrics and policy design.
	
	What Counts as Boilerplate
	
	simulator/models.py: Type definitions (TypedDict) and dataclasses defining parcels, configuration, contexts, and event records.
	
	simulator/loaders.py: File-reading routines that parse and validate input files (demand/*.csv, dist.csv, config.json).
	
	simulator/simulator.py: The underlying simulation engine, event queue ordering, route validation, and log.txt writing logic.
	
	performance_monitor.py CLI logic: The command-line argument parsing, file reading, and JSON report generation logic inside main().
	
	calculate_metrics Function Skeleton
	
	Here is a basic function skeleton for calculate_metrics in performance_monitor.py with placeholders for custom metrics:
	
	`def calculate_metrics(parcels, distances, config, events):`
	`    """Process log events and simulation data to compute user-defined metrics.`
	
	`Args:`
	`parcels (list[dict]): Raw parcel records loaded from the demand CSV.`
	`distances (list[dict]): Distance records loaded from dist.csv.`
	`config (dict): Vehicle and fleet settings from config.json.`
	`events (list[dict]): Sequence of simulation events loaded from log.txt.`
	
	`Returns:`
	`dict: A JSON-serializable dictionary containing your metrics.`
	`"""`
	`# 1. Initialize metric tracking variables`
	`# (e.g., metric_a = 0, metric_b = [])`
	
	`# 2. Iterate through events or parcel data`
	`for event in events:`
	`event_type = event.get("event_type")`
	
	`# Add conditional checks for specific event types if needed`
	`# pass`
	
	`# 3. Compute final metric values`
	`# (e.g., averages, counts, rates)`
	
	`# 4. Return metric key-value pairs`
	`return {`
	` "custom_metric_1": None,`
	` "custom_metric_2": None,`
	`}`
	
	You are doing a fantastic job exploring the codebase structure—feel free to reach out if you have any further questions about how the simulator components interact!

Can you show a basic dispatch function skeleton using the required imports, parameters and return type? Leave the parcel selection and route ordering for us to implement.	Here is the function skeleton for your custom policy file (e.g., solution/policies/custom_policy.py), using the project's type definitions:
	`from typing import Dict, List`
	`from simulator.models import Parcel, SimulationContext`
	
	`def dispatch(parcel: Dict[str, Parcel], context: SimulationContext) -> List[str]:`
	`"""Custom dispatch policy entry point.`
	
	`Args:`
	`parcel: Map of parcel_id to Parcel dictionary records.`
	`context: SimulationContext containing current_time, vehicle_id,`
	`travel_distance, capacity, max_trip_distance_km, etc.`
	
	`Returns:`
	`List[str]: Ordered sequence of parcel IDs to deliver on this trip.`
	`Return an empty list [] if no trip should be dispatched.`
	`"""`
	`route: List[str] = []`
	
	`# Implement parcel selection and route ordering logic here`
	
	`return route`
	
	You are setting up your workspace effectively—best of luck developing your custom dispatch policy!